

Bước đầu tìm hiểu thuật toán không hoàn hảo

Ren Yiheng

Trường Trung học số 1 Đường Sơn, Hà Bắc

Trại đông Tin học toàn quốc, 2008

Nguồn gốc: Ứng dụng của thuật toán không hoàn hảo

IOI China candidate team papers / 2008 / Day 2 / Ren Yiheng / imperfect algorithms.doc

Tóm tắt

Trong luyện tập và thi đấu, ta thường cố thiết kế thuật toán hoàn toàn đúng. Nhưng trong thực tế có nhiều bài toán không thể giải hoàn hảo trong giới hạn thời gian và bộ nhớ cho phép; cũng có những bài chấm theo chất lượng nghiệm, trong đó lời giải gần tối ưu vẫn có giá trị. Bài viết giới thiệu một số thuật toán không hoàn hảo thường dùng: tham lam, ngẫu nhiên, kiểm thử bằng lấy mẫu và mô phỏng luyện kim, rồi minh họa bằng các bài thi cụ thể.

Từ khóa: thuật toán không hoàn hảo; tham lam; ngẫu nhiên; lấy mẫu; mô phỏng luyện kim.

1 Dẫn nhập

Trong các bài toán chuẩn, mục tiêu tự nhiên là tìm thuật toán đúng tuyệt đối. Tuy nhiên, nhiều bài toán thực tế quá lớn hoặc quá khó để giải tối ưu. Một số bài trong thi tin học cũng đã tiến gần hơn tới thực tế: lời giải được chấm theo độ tốt của kết quả, hoặc theo số lượng trường hợp trả lời đúng trong giới hạn thời gian. Khi đó, một thuật toán không hoàn hảo nhưng nhanh, đơn giản và cho nghiệm tốt có thể hữu ích hơn một thuật toán lý thuyết đẹp nhưng không thể cài xong hoặc không chạy nổi.

“Không hoàn hảo” ở đây không có nghĩa là tùy tiện. Ta vẫn cần hiểu cấu trúc bài toán, biết thuật toán có thể sai ở đâu, và cố làm cho kết quả gần tối ưu nhất có thể. Các phần sau trình bày bốn nhóm kỹ thuật qua ví dụ.

2 Thuật toán tham lam

Ý tưởng cơ bản của tham lam là xuất phát từ một trạng thái ban đầu rồi liên tục chọn bước có vẻ tốt nhất hiện tại, nhằm nhanh chóng tiến tới mục tiêu. Khi không còn bước cải thiện rõ ràng, thuật toán dừng. Ta nhận được một lời giải, nhưng không nhất thiết là lời giải tối ưu.

2.1 Ví dụ 1: NOI 2007, Truy bắt kẻ trộm

Một quốc gia cần truy bắt một tên trộm trên mạng thành phố có dạng cây. Cảnh sát có thể được thả xuống một thành phố, di chuyển dọc cạnh, hoặc được thu hồi. Cần lập kế hoạch sao cho dù tên trộm ở đâu và di chuyển thế nào, hẳn chắc chắn bị bắt; số cảnh sát dùng càng ít càng tốt.

Thuật toán chuẩn của bài này dùng nhiều kiến thức nâng cao và hiện thực khá phức tạp. Trong thời gian thi, giải hoàn hảo là nhiệm vụ nặng. Một chiến lược tham lam đơn giản là xem cây như cây có gốc. Ta thả một cảnh sát ở gốc để canh, rồi thả thêm một cảnh sát ở gốc để lần lượt đi vào các cây con và loại trừ từng nhánh. Khi chỉ còn một cây con chưa xử lý, không cần thả cảnh sát mới nữa; cảnh sát đang canh ở gốc có thể đi vào nhánh cuối cùng. Vì vậy nên để cây con cần nhiều cảnh sát nhất xử lý sau cùng.

Với $n \leq 1000$, có thể thử từng đỉnh làm gốc, chạy chiến lược trên và chọn kết quả tốt nhất. Thuật toán có phần ví dụ, nhưng tối ưu “để nhánh khó nhất sau cùng” làm kết quả rất gần nghiệm chuẩn. Theo thí nghiệm trong bài gốc, khoảng 90% kết quả trùng với thuật toán chuẩn; phần còn lại cũng chỉ lệch nhỏ.

2.2 Cải thiện tham lam

Ưu điểm của tham lam là dễ cài và chạy nhanh. Nhược điểm là độ chính xác có thể kém khi chiến lược cục bộ bỏ qua thông tin toàn cục. Có hai hướng cải thiện thường gặp:

- kết hợp tham lam với một thuật toán đúng trên từng phần nhỏ, rồi dùng tham lam để ghép các kết quả cục bộ;
- chọn hàm đánh giá tham lam chứa nhiều thông tin ảnh hưởng tới toàn cục hơn, thay vì chỉ nhìn một tiêu chí ngắn hạn.

3 Thuật toán ngẫu nhiên

Ngẫu nhiên cũng thường xuất hiện trong các thuật toán không hoàn hảo. Nó hiếm khi đứng một mình; thường nó được ghép với tham lam, tìm kiếm cục bộ hoặc một thuật toán nền để tránh kẹt trong cấu hình xấu.

3.1 Ví dụ 2: Yet Another Computer Network Problem

Cho đồ thị vô hướng n đỉnh, m cạnh. Cần xây một cây khung có tổng trọng số nhỏ, đồng thời bậc mỗi đỉnh không vượt quá giới hạn b . Nếu bỏ điều kiện bậc, đây là bài cây khung nhỏ nhất đơn giản; điều kiện bậc làm bài toán khó hơn nhiều.

Một cách làm thực dụng là trước hết xây một cây gần với cây khung nhỏ nhất nhưng tôn trọng giới hạn bậc nhiều nhất có thể, chẳng hạn biến thể của Kruskal. Sau đó dùng cải thiện ngẫu nhiên: chọn ngẫu nhiên một đỉnh đã đạt giới hạn bậc, chọn một cạnh đang thuộc cây và một cạnh ngoài cây liên quan tới đỉnh đó để thử hoán đổi. Sau khi thêm cạnh mới, cây tạo thành một chu trình; ta xóa cạnh dài nhất trên chu trình. Nếu tổng trọng số hoặc điểm đánh giá tốt hơn, giữ thay đổi.

Quy trình này là một tìm kiếm cục bộ ngẫu nhiên. Nó không bảo đảm tối ưu, nhưng trong bài gốc cho kết quả rất tốt vì mỗi bước đều nhắm vào những đỉnh đang gây áp lực lên ràng buộc bậc.

4 Kiểm thử bằng lấy mẫu

Lấy mẫu là chọn một phần tử đại diện từ tổng thể rồi suy ra tính chất của tổng thể. Trong thuật toán, kỹ thuật này dùng được khi tập kiểm tra có tính chất: hoặc không có phần tử tốt nào, hoặc có rất nhiều phần tử tốt. Khi đó chỉ cần thử một số mẫu nhỏ đã có xác suất hoặc bằng chứng đủ mạnh.

4.1 Ví dụ 3: Prime Checker

Bài SPOJ 919 yêu cầu kiểm tra càng nhiều số càng tốt trong một dãy truy hồi cho trước, mỗi số cần in ra là nguyên tố hay hợp số. Cách cơ bản là sàng các số nguyên tố nhỏ rồi thử chia từng a_i bởi các số nguyên tố không vượt quá $\sqrt{a_i}$. Cách này rất dễ viết nhưng trong giới hạn thời gian chỉ xử lý được hơn hai trăm nghìn số.

Một lựa chọn tốt hơn là kiểm tra Miller–Rabin. Với số lẻ n , viết

$$n - 1 = d2^s, \quad d \text{ lẻ.}$$

Với một cơ số a , nếu

$$a^d \equiv 1 \pmod{n}$$

hoặc tồn tại $0 \leq r < s$ sao cho

$$a^{d2^r} \equiv -1 \pmod{n},$$

thì n vượt qua phép thử mạnh theo cơ số a . Mọi số nguyên tố đều vượt qua với mọi cơ số hợp lệ. Với hợp số, nếu chọn cơ số ngẫu nhiên, xác suất bị đánh lừa không vượt quá $1/4$ cho mỗi lần thử độc lập.

Trong bài này, ta không nhất thiết chọn ngẫu nhiên. Thử nghiệm cho thấy:

- chỉ dùng cơ số 2, phản ví dụ đầu tiên là 2047;
- dùng 2, 3, phản ví dụ đầu tiên là 1373653;
- dùng 2, 3, 5, phản ví dụ đầu tiên là 25326001;
- dùng 2, 3, 5, 7, mọi số trong phạm vi 2^{31} của bài đều đủ an toàn.

Nhờ vậy bài toán chuyển từ thử chia nhiều ước sang vài phép lũy thừa modulo, độ phức tạp xấp xỉ $\mathcal{O}(N \log N)$ theo số phần tử cần kiểm tra, nhanh hơn rất nhiều trong thực tế.

5 Mô phỏng luyện kim

Mô phỏng luyện kim bắt nguồn từ quá trình ủ kim loại: nung nóng vật rắn rồi làm nguội dần, để hệ có cơ hội thoát khỏi trạng thái năng lượng cao và cuối cùng về trạng thái năng lượng thấp. Trong tối ưu tổ hợp, ta xem giá trị mục tiêu là năng lượng và nhiệt độ là tham số điều khiển.

Quy trình chung:

1. chọn nghiệm ban đầu S , nhiệt độ ban đầu T đủ lớn, và số lần lặp ở mỗi mức nhiệt;
2. sinh nghiệm lân cận S' ;
3. tính $\Delta = C(S') - C(S)$, với C là hàm đánh giá;
4. nếu $\Delta < 0$, nhận S' ; nếu không, nhận S' với xác suất $\exp(-\Delta/T)$;
5. giảm dần T , lặp cho tới khi đạt điều kiện dừng.

Khác với tìm kiếm cục bộ thuần túy, mô phỏng luyện kim thỉnh thoảng chấp nhận nghiệm xấu hơn, đặc biệt khi nhiệt độ còn cao. Điều này giúp thuật toán thoát khỏi cực trị địa phương.

5.1 Ví dụ 4: TSP

Trong bài người bán hàng du lịch, cần tìm chu trình đi qua n thành phố đúng một lần với tổng độ dài nhỏ nhất. Không gian nghiệm là các hoán vị vòng $(w_1, w_2, \dots, w_n)$, với $w_{n+1} = w_1$. Hàm mục tiêu là

$$\sum_{i=1}^n d(w_i, w_{i+1}).$$

Một cách sinh nghiệm lân cận là chọn hai vị trí k, m rồi đảo đoạn giữa chúng, hoặc đảo hai đầu tùy thứ tự. Hiệu chi phí có thể tính từ các cạnh bị thay đổi thay vì tính lại toàn bộ chu trình. Đưa cách sinh này vào khung mô phỏng luyện kim sẽ cho một lời giải gần tối ưu cho TSP.

Khi dùng mô phỏng luyện kim, ba tham số cần chú ý là:

- nhiệt độ ban đầu: quá thấp dễ kẹt, quá cao tốn thời gian;
- tốc độ giảm nhiệt: giảm nhanh thì chạy nhanh nhưng tìm kiếm kém rộng;
- số lần lặp ở mỗi nhiệt độ: cần đủ để hệ gần cân bằng ở mức nhiệt đó.

Trong thực tế thường dùng

$$T_{t+1} = kT_t,$$

với k là hằng số dương nhỏ hơn 1 một chút.

6 Thực hành: Baidu Star 2007, Emergency Repair

Một thành phố có k công ty bị tê liệt. Sau T giờ hệ thống tự động sẽ sửa xong, nhưng trước đó mỗi công ty gây thiệt hại P_i mỗi giờ. Có n đội sửa chữa di chuyển trên lưới $R \times C$, mỗi giờ đi được tối đa s ô hoặc sửa công ty tại vị trí hiện tại. Cần lập kế hoạch để tổng thiệt hại nhỏ nhất.

Bài này không có thuật toán hoàn hảo đơn giản. Một hướng thực dụng là:

1. chạy BFS để tính khoảng cách từ các vị trí cần thiết tới từng công ty; vì mỗi hành trình kết thúc ở một công ty, không cần mọi cặp ô;
2. chọn một thứ tự sửa các công ty;
3. với công ty hiện tại, nếu một đội sửa chữa có thể tới đó và làm giảm thời gian hoàn thành, điều đội ấy tới sửa; nếu không, để đội chuyển sang mục tiêu sau;
4. dùng mô phỏng luyện kim để tối ưu thứ tự công ty.

Bước BFS có độ phức tạp $\mathcal{O}(RCK)$. Phần khó là thứ tự công ty; mô phỏng luyện kim cung cấp một cách tìm thứ tự tốt mà không phải duyệt toàn bộ hoán vị.

7 Kết luận

Các thuật toán không hoàn hảo có phạm vi ứng dụng rộng. Chúng thường tiêu tốn ít thời gian và bộ nhớ, dễ nghĩ và dễ cài hơn thuật toán tối ưu hoàn toàn. Trong nhiều bài chấm theo chất lượng nghiệm, hoặc trong những tình huống mà thuật toán chính thống quá phức tạp, chúng có thể cho kết quả không kém hoặc thậm chí tốt hơn về mặt thực dụng.

Điều quan trọng là dùng chúng một cách có kiểm soát: hiểu rằng buộc nào được đảm bảo, phần nào chỉ dựa vào kinh nghiệm, và có thể cải thiện nghiệm bằng tham lam cục bộ, ngẫu nhiên, lấy mẫu hay mô phỏng luyện kim ra sao.

Tài liệu tham khảo

1. Yang Pei, *Bước đầu tìm hiểu thuật toán phi tối ưu*, luận văn năm 2000.
2. Hu Weidong, *Phân tích ứng dụng thuật toán không hoàn hảo trong thi tin học*, luận văn năm 2005.
3. Lou Tiancheng, báo cáo lời giải Baidu Star.
4. Tài liệu về *simulated annealing algorithm*.

Ghi chú về phụ lục nguồn

DOC gốc kèm toàn văn các đề *Truy bắt kẻ trộm*, *Prime Checker*, *Yet Another Computer Network Problem* và *Emergency Repair*. Bản dịch đã rà lại các phần tham lam, ngẫu nhiên, lấy mẫu, Miller–Rabin, mô phỏng luyện kim, TSP và *Emergency Repair*; không còn tham chiếu hình hoặc chú thích treo. Các phát biểu dài và tài liệu chương trình trong nguồn được giữ theo ngoại lệ phụ lục/mã mẫu: không chép nguyên văn, chỉ đưa những chi tiết cần cho phân tích vào các ví dụ tương ứng ở trên.